Tempo: Effect-Based Synchronous Reactive Control in OCaml

Frédéric Dabrowski
Univ. Orléans, INSA Centre Val de Loire, LIFO EA 4022
Orléans, France
frederic.dabrowski@univ-orleans.fr

ABSTRACT

Tempo is a lightweight synchronous runtime for OCaml that revisits the reactive control lineage of Esterel, FairThreads, and ReactiveML using OCaml 5.3 algebraic effects. The library implements logical instants, valued signals, guards, and weak preemption while preserving the delayed-absence discipline that enables deterministic concurrency under standard purity assumptions (no shared mutable state dependent on intra-instant order and order-insensitive aggregation). This report positions Tempo in its semantic context, explains its API and usage patterns, and provides a detailed account of how effect handlers realize the scheduler and control primitives at library level. The current draft emphasizes design and implementation; formal correctness and quantitative evaluation are identified as future work.

KEYWORDS

tempo, ocaml, synchronous, reactive control, algebraic effects

ACM Reference Format:
Frédéric Dabrowski. 2026. Tempo: Effect-Based Synchronous Reactive Control in OCaml. In *Proceedings of ACM Conference (Conference'17).* ACM, New York, NY, USA, 18 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Reactive systems must continuously respond to an environment, often under strict timing and safety constraints. Programming such systems is difficult because concurrency, external events, and state updates interact in ways that can break determinism. Ad-hoc callback-based designs are common, but they make control flow implicit and complicate reasoning about ordering, causality, and reproducibility.

Synchronous reactive programming addresses these challenges by discretizing time into logical instants and by defining what is observable at instant boundaries. A classic distinction is between control-flow and dataflow approaches: Esterel represents the control-flow lineage, while Lustre represents the dataflow lineage [9, 19]. In both cases, logical instants enable deterministic behavior and rigorous analyses of causality and scheduling. Survey discussions situate these languages within a shared semantic framework [6].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA
© 2026 Association for Computing Machinery.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

These synchronous languages achieve high assurance by imposing restrictions that permit static scheduling and causality checks. The restrictions make it possible to decide properties such as determinism and absence/presence consistency, but they also limit expressiveness when integrated with richer general-purpose languages.

The key semantic question is when signal absence can be observed. In Esterel, absence is decided within the instant, enabling static scheduling but restricting dynamic behavior [9]. Boussinot's FairThreads model defers absence observation to instant end, allowing more flexible composition while preserving determinism [10]. From a programmer's perspective, each instant follows a two-phase discipline: react while tasks run and signals may become present, then finalize when the instant quiesces and absence becomes observable. This distinction is central because it determines which control patterns are expressible without sacrificing determinism. Subsequent work revisits the synchronous model inside richer language settings. On the control-flow side, ReactiveML adapts Esterel-style signals and the synchronous hypothesis to a higher-order language [23]. On the dataflow side, FRP explores declarative, time-varying values with different operational priorities [17].

This paper introduces Tempo: a library that revisits ReactiveML's control-flow model using OCaml 5.3 algebraic effects as its implementation mechanism. Tempo follows the delayed-absence discipline introduced by Boussinot's FairThreads, which ReactiveML also adopts, and brings this approach to a modern OCaml runtime without modifying the compiler [10, 23]. Tempo re-implements the core control constructs (signals, guards, weak preemption) as a pure library. It executes programs in discrete logical instants: signals can be emitted and tasks can run within an instant, but the *absence* of a signal is only observable at the end of the instant. This article presents Tempo as a reactive control runtime and explains its design choices and effect-based implementation in OCaml 5.3. It does not attempt a formal semantics or a quantitative evaluation, and focuses instead on the design rationale, API, and implementation details needed to motivate future formalization. Determinism is discussed under standard purity assumptions: programs should avoid shared mutable state whose outcomes depend on intra-instant scheduling, and aggregate combine functions should be order-insensitive. These assumptions match the synchronous control model and are stated explicitly in the core constructs section.

- A library-level formulation of synchronous reactive control in OCaml, positioned within the Esterel/FairThreads/ReactiveML lineage.
- An implementation account of how algebraic effects realize signals, guards, suspension, and weak preemption.
- End-to-end examples that illustrate standard synchronous control idioms using the Tempo API.

2 BOUSSINOT'S REACTIVE CONTROL MODEL

Boussinot's reactive control model organizes concurrency around logical instants. Execution proceeds as a sequence of reaction cycles: within an instant, all runnable activities execute until they suspend or the system quiesces, and only after that quiescence can absence be observed. This notion of instant provides a global time base that makes reactions comparable and deterministic at instant boundaries [10].

Signals (unvalued). To simplify the presentation, we consider unvalued signals. A signal is either present or absent for the entire duration of an instant. When a signal is emitted, it becomes present for all components; when it is not emitted, it remains absent for all components. The effective order of emissions within the instant must not affect the observable behavior: presence is a global property for the instant, not a per-thread event. This makes synchronization explicit and avoids order-sensitive communication.

Suspension. Threads are cooperative: they execute until they reach a suspension point and then yield control. Suspension can be explicit (e.g., waiting for a signal or for the next instant). The key property is that suspension preserves the instant structure: a suspended thread resumes only at a well-defined instant boundary or when a relevant signal becomes present.

Algorithmic view (abstract). We use a small algorithmic notation that mirrors Boussinot's control vocabulary. Behaviors (threads) are composed from a core set of control operators. The notation hides values when they are not essential to control and uses abstract behaviors inside control structures:

```
(* Abstract control language; B is an abstract behavior. *)
B ::= skip
    | emit s (* mark signal s present in current instant *)
    | await s (* resume at next instant where s is present
        *)
    | pause (* resume at the next instant *)
    | B1; B2 (* sequence *)
    | parallel [B1; ...; Bn]
    | when s do B (* guard: block until s is present *)
    | watch s do B (* weak preemption: stop B after this
        instant if s emits *)
```

The constructs are interpreted at instant boundaries. `emit s` makes `s` present for the remainder of the instant. `await s` suspends until the next instant where `s` is present, while `pause` always advances to the next instant. In `when s do B`, the guard is a blocking condition: if `s` is absent, the entire behavior suspends and can only resume at a future instant where `s` is present. In `watch s do B`, the body `B` runs normally in the current instant, but if `s` is emitted during that instant, `B` is not allowed to continue into the next instant (weak preemption).

This abstract view lets us express control patterns without committing to a concrete evaluation order. For example, `when s do {B1; B2}` denotes an entire guarded behavior, and `watch s do {parallel [B1; B2]}` denotes a preemptible concurrent block.

```
Micro-examples (abstract)
(* Suspension across instants *)
emit s; pause; await s

(* Guarded behavior: runs only if s is present *)
when s do (emit out)

(* Weak preemption: body stops after instant if k is emitted
    *)
watch k do (emit tick; pause; emit tick)

(* Disjunctive enabling via parallel guards *)
parallel [
  (when g1 do B);
  (when g2 do B)
]
```

Preemption and Esterel. Control languages such as Esterel rely on preemption to enforce control structure. In Boussinot's setting, preemption is *weak*: if a preempting signal is emitted in instant t, a preempted computation may complete its current instant but must not continue into $t+1$. The alternative would be *strong* preemption, where a computation is stopped immediately within the same instant. Weak preemption is the appropriate choice here because the model treats the instant as an atomic reaction: all threads share the same present/absent view for the whole instant, and mid-instant interruption would reintroduce order dependence and break determinism [9, 10].

This model yields a control-centric view of concurrency: threads cooperate, rendezvous on signal presence, and advance in lockstep from one instant to the next. Tempo adopts this discipline as the semantic baseline for its runtime.

From FairThreads to ReactiveML (abstract view). ReactiveML lifts this control model into a higher-order ML setting while preserving the delayed-absence discipline. It exposes a small set of reactive operators—`signal`, `emit`, `await`, `pause`, `when`, and `watch`—that directly encode the instant-based control structure. These constructs compile to a runtime that enforces the instant discipline and yields deterministic behavior at instant boundaries, while allowing higher-order composition over reactive processes [23].

At the same level of abstraction, this places ReactiveML in the control-flow lineage (Esterel/FairThreads), in contrast to dataflow approaches such as FRP that emphasize time-varying values and stream composition [17]. Recent FRP work has refined operational guarantees such as causality and space usage (e.g., [2–4, 27]), but these advances are orthogonal to the control-centric lineage considered here.

ReactiveML also benefits from language-level integration: its compiler can enforce discipline constraints and optimize the scheduling structure statically. Tempo chooses a different point in the design space by reusing OCaml 5.3 algebraic effects to recover similar control constructs as a library. This enables straightforward embedding in existing OCaml codebases while leaving stronger static guarantees to future work.

3 REACTIVEML CONTEXT AND CORE CONSTRUCTS

ReactiveML is the closest language-level incarnation of the control model described in Section 2. It offers dedicated syntax, compiler support, and a runtime tailored to instants and delayed absence [23]. We do not reproduce the language itself here; instead, we use it as a point of reference for the library-based approach of Tempo.

The remainder of this section therefore focuses on the core control constructs and their synchronous properties, as they appear in Tempo's library API. Tempo exposes a small set of constructs that mirror the common vocabulary of reactive control languages. Signals are the primary communication medium within an instant. A signal is either present or absent during an instant; presence is established by emission, while absence can only be observed at the end of the instant. This delayed observation is central to deterministic behavior.

Semantic contract (informal). Tempo guarantees determinism at instant boundaries: for a fixed input stream, the set of signals present at each instant and the values delivered to awaiters are deterministic. What is not specified is the internal evaluation order of tasks within an instant; relying on that order can break determinism when using shared mutable state. The semantics therefore define *what* is observable per instant, not the precise scheduling order inside it. We use *instant boundary* and *end of the instant* interchangeably.

Non-guarantees. Tempo does not guarantee deterministic outcomes when programs depend on the intra-instant evaluation order or shared mutable state (e.g., reference cells). Such behaviors are outside the semantic contract and must be avoided or controlled by static analyses.

Minimal vocabulary.

- **Instant**: logical time step where reactions occur.
- **Presence/absence**: a signal is present if emitted in the instant.
- **Guard**: a condition that blocks a task until a signal is present.
- **Weak preemption**: termination at the end of the instant.
- **Determinism**: observables per instant are uniquely determined by inputs.

Determinism statement (informal). Let I be a fixed input sequence per instant. Tempo guarantees that, for each instant t, the multiset of event-signal emissions and the values returned by `await` and `await_immediate` are uniquely determined by I, assuming programs avoid shared mutable state that depends on intra-instant ordering and aggregate combine functions are pure and order-insensitive. No guarantee is made about the relative order of tasks within an instant.

Observable vs non-observable (summary).

- **Observable:** which signals are present at each instant boundary, and the values delivered to `await` and `await_immediate`.
- **Not observable:** the order in which tasks run within the instant, or the ordering of side effects on shared mutable state.

Aggregate signals and `await_immediate`. Aggregate signals are delivered only at instant end; thus their determinism is defined by the combined value, not by the order of emissions within the instant. `await_immediate` is restricted to event signals, which allows safe same-instant resumption without introducing ambiguity in the delivered value. For deterministic aggregation, the combine function should be pure and order-insensitive (associative/commutative), since the intra-instant order of emissions is not part of the semantics.

Observable trace (informal). An execution can be summarized by a per-instant trace $au(t)$ that records:

- **Event signals:** the unique value emitted in instant t, or *absent* if no emission occurs.
- **Aggregate signals:** the combined value delivered at instant end (or the initial accumulator if no emission occurred).
- **Outputs:** values emitted on the output signal at instant end.

The trace intentionally omits intra-instant scheduling order. Two runs are observationally equivalent if they produce the same $au(t)$ for all t.

`await` *vs* `await_immediate`. `await` always resumes at the next instant, even if the signal is already present. If the event signal is present, `await_immediate` may resume in the current instant. This can be useful for immediate reactions, but it is restricted to event signals to preserve determinism.

Observable per instant:
1) set of present signals
2) values delivered to awaiters
3) outputs emitted to the environment

Figure 1: Observables that define determinism at instant boundaries.

Determinism in action. For the same input stream, the observables above do not change. For example, if the host emits input 1 at instant t and 2 at instant t+1, then the program below deterministically emits 1 at t+1 and 2 at t+2.

```
let program input output =
  let v = await input in
  emit output v
```

Instant	Observable events
t	input emits 1; no output yet.
t+1	output emits 1; input emits 2.
t+2	output emits 2.

Table 1: Timeline for the simple echo program.

Mini-specification (informal). The behavior of a program can be described per instant:

(1) At instant start, all signals are absent.
(2) Emissions during the instant mark signals present; event signals accept at most one emission.
(3) `await` resumes at the next instant; `await_immediate` may resume in the current instant for event signals already present.
(4) Absence is observed only at instant end; guard rechecks occur at the next instant.

Intuitive timeline. Consider two instants with a single signal `s`. In instant t, a task emits `s` and another task awaits it. The awaiting task can only resume at the *beginning* of instant t+1, because the presence of `s` becomes observable only after the instant ends. If `s` is never emitted in instant t, that absence is observable only at the end of t, and a guarded task can only retry in instant t+1. Observable events are therefore the set of signals present at instant boundaries, not intermediate scheduling steps within the instant.

Emissions mark a signal as present and provide a value. Tempo supports two signal kinds: event signals accept at most one emission per instant, while aggregate signals combine multiple emissions through an associative folding function. Suspension is expressed through `await`, which resumes at the beginning of the next instant, and `await_immediate`, which may resume in the current instant if an event signal is already present.

Guards and preemption encode control flow. Guarded execution activates a behavior only when a guard signal becomes present. Weak preemption terminates a behavior at the end of the current instant after a preempting signal is emitted, allowing other concurrent tasks to observe the emission within the same instant. These constructs preserve determinism by maintaining a single global notion of instants and deferring absence decisions to instant boundaries. The distinction between strong and weak restart patterns is often encoded by explicit pauses: inserting a `pause` between successive activations ensures that a reaction cannot retrigger within the same instant, which matches the weak restart discipline commonly used in synchronous control.

Guarded execution is *blocking*, so a sequential nesting of `when_` does not implement a disjunction. A disjunction is obtained by running multiple guarded branches in parallel, each able to unblock independently. Similarly, nested `watch` constructs compose as disjunctions: a body is preempted if any watched signal is emitted in the current instant. This is consistent with the control interpretation where multiple guards or preemption sources represent alternative enabling or disabling conditions.

```
(* Disjunction of guards: run guarded branches in parallel.
     *)
parallel [
  (fun () -> when_ g1 (fun () -> body ()));
  (fun () -> when_ g2 (fun () -> body ()));
];

(* Disjunction of preemption: stop when s1 or s2 is emitted.
      *)
watch s1 (fun () -> watch s2 (fun () -> body ()))
```

4 TEMPO OVERVIEW

Tempo is a lightweight synchronous runtime for OCaml. It adopts the delayed absence principle of FairThreads and ReactiveML, but implements it entirely at library level using OCaml 5.3 algebraic effects. Programs execute over a sequence of logical instants, interact via valued signals, and compose concurrently under a deterministic scheduler.

The runtime preserves the core control semantics: signal presence is decided within the instant, absence is observed only at instant end, and preemption is weak and instant-aligned. The key design choice is to encode these constructs as effectful operations interpreted by handlers. This keeps the model explicit in user code and avoids changes to the language runtime, while still enabling expressive control operators and a clear mapping to synchronous concepts.

Compared to ReactiveML, Tempo trades language-level syntax and compilation support for a pure library approach based on OCaml 5.3 effects, making it easier to embed into existing codebases and to experiment with scheduling.

Programming model. Tempo programs are written as ordinary OCaml functions that receive two signals: an input signal provided by the host and an output signal used to publish results at the end of each instant. Instants are advanced by the `execute` loop, which invokes host hooks before and after each instant. Signals are first-class values, so they can be stored in data structures, returned from functions, and passed between components.

Determinism and control. Determinism is defined at instant boundaries: given the same input emissions, the observable outputs at each instant are fixed, regardless of internal task interleavings. Within an instant, the scheduler runs tasks in a fixed order, but the semantics does not expose this ordering, which keeps the model declarative from the user's perspective.

Design rationale. Tempo adopts algebraic effects for three reasons. First, effects provide first-class, delimited continuations, which allow the scheduler to control resumption points precisely. Second, a library-level approach avoids modifying the OCaml compiler and keeps integration friction low. Third, handlers make the control flow explicit in user space, which simplifies experimentation with scheduling policies and debugging.

Non-goals (current scope). Tempo does not attempt to provide static guarantees (e.g., single-emission checks or determinism proofs) and does not perform compilation into automata. These choices keep the library small and explainable, but they also motivate the future work discussed in Section 12.

5 PUBLIC API

Tempo exposes a small API centered on signals, instants, and control combinators. The API can be grouped into three layers: signal creation and manipulation, control flow within instants, and execution of a complete reactive program.

Signal types. Tempo uses two signal families:

- `'a signal`: event signal carrying values of type `'a`. At most one emission is allowed per instant.

- `('emit, 'agg) agg_signal`: aggregate signal that can receive multiple emissions of type `'emit` per instant, folded into a value of type `'agg` using a user-provided combine function.

Informal type signatures.

- `new_signal : unit -> 'a signal`
- `new_signal_agg : initial:'agg -> combine:('agg -> 'emit -> 'agg) -> ('emit, 'agg) agg_signal`
- `emit : 'a signal -> 'a -> unit`
- `await : 'a signal -> 'a`
- `await_immediate : 'a signal -> 'a`
- `pause : unit -> unit`
- `when_ : 'a signal -> (unit -> unit) -> unit`
- `watch : 'a signal -> (unit -> unit) -> unit`
- `parallel : (unit -> unit) list -> unit`
- `fork : (unit -> unit) -> thread`
- `join : thread -> unit`
- `execute : ('a signal -> 'b signal -> unit) -> unit`

These signatures are indicative; see Tempo for precise types.

Signal creation and communication.

- `new_signal ()` creates an event signal.
- `new_signal_agg extasciitilde initial extasciitilde combine` creates an aggregate signal with an initial accumulator and a combine function.
- `emit signal value` marks a signal present in the current instant and delivers `value`.
- `await signal` suspends the current task until the signal becomes present, resuming at the beginning of the next instant with the signal value.
- `await_immediate signal` resumes in the same instant if the signal is already present. To preserve determinism, it is restricted to event signals.

```
let s = new_signal ()
emit s 1
let v = await s
```

Aggregate signals support accumulation across multiple emissions in the same instant:

```
let s = new_signal_agg
  ~initial:0
  ~combine:(fun acc x -> acc + x)
emit s 1;
emit s 2;
let sum = await s
```

Contract. The combine function should be pure and ideally associative and commutative; otherwise the aggregate result may depend on the intra-instant evaluation order, which is not specified by the semantics. If no emission occurs during an instant, the value delivered is the initial accumulator.

Control flow and concurrency.

- `pause ()` suspends the current task until the next instant.
- `parallel [p1; ...; pn]` executes a list of behaviors synchronously within the same instant.
- `when_ guard body` executes `body` only if `guard` is present; otherwise the task is blocked until the guard becomes present.
- `watch signal body` executes `body` and terminates it at the end of the instant in which `signal` is emitted (weak preemption).
- `fork` and `join` create and synchronize logical threads.

```
parallel [
  (fun () -> emit s 1);
  (fun () -> pause (); emit s 2)
]
watch s (fun () ->
  when_ s (fun () -> emit s 3))
```

Usage notes:

- Event signals enforce single emission per instant; aggregate signals are designed for multi-producer scenarios.
- `await` always resumes in the next instant, while `await_immediate` can resume in the current instant if the signal is already present.
- `parallel` defines a logical concurrency scope; `fork` and `join` are useful when a thread handle is needed.

Execution entry point.
`execute` runs a program over instants. It accepts optional `input` and `output` hooks to interact with the environment by injecting values into an input signal and observing an output signal at each step. Inputs are injected at the beginning of each instant; outputs are observed at the end of the instant after the scheduler reaches quiescence. The program is expressed as a function that receives these two signals and defines the reactive behavior.

```
execute (fun input output ->
  let v = await input in
  emit output v)
```

Low-level API. The `Tempo.Low_level` module exposes explicit termination and thread-control primitives. It is intended for advanced uses, such as building custom control operators that are not directly provided by the high-level API.

Summary table.

6 USAGE AND EXAMPLES

This section provides a minimal example that emits a signal and awaits its presence in the next instant, followed by a larger example that uses guarded execution and weak preemption. The examples are aligned with the tests in `tests/ok` to ensure they are executable and representative (e.g., `emit_once_await_one.ml`, `watch_when_complete.ml`).

Item	Role and constraints
`'a signal`	Event signal; at most one emission per instant.
`('e,'a) agg_signal`	Aggregate signal; combine multiple emissions.
`emit`	Mark present; event signals accept a single emit per instant.
`await`	Suspend to next instant; resumes with signal value.
`await_immediate`	Same-instant await; event signals only.
`pause`	Suspend until next instant.
`when_`	Guarded execution; blocks until guard is present.
`watch`	Weak preemption; stops at end of emitting instant.
`execute`	Run program; host hooks optional.

Table 2: API summary with roles and constraints.

To execute the examples, run the corresponding test binaries, e.g.:

```
dune exec tests/ok/emit_once_await_one.exe
dune exec tests/ok/watch_when_complete.exe
```

Reading the examples. Each example is written as a function `program` passed to `execute`. The host provides an input stream by emitting on the input signal before each instant and observes output after each instant. Thus, the observable behavior of a program is a sequence of values emitted on the output signal at instant boundaries.

End-to-end example 1: input/output echo. This example waits for an input value and echoes it on the output signal in the next instant.

```
let program input output =
  let v = await input in
  emit output v

let () = execute program
```

Step-by-step behavior:

(1) Instant t: the host emits on `input` via the `execute` hook; the program is awaiting and either registers a waiter or is scheduled for the next instant if the signal is already present.
(2) Instant t+1: the awaiting continuation resumes and emits on `output`, which the host observes at the end of the step.

End-to-end example 2: guarded wait with timeout. This example waits for a request, but is weakly preempted if a timeout signal is emitted after n instants.

```
let rec timeout_after n s =
  if n = 0 then emit s ()
  else (pause (); timeout_after (n - 1) s)

let program input output =
  let request = new_signal () in
  let timeout = new_signal () in
  let _ = fork (fun () -> timeout_after 3 timeout) in
  let handled = new_signal () in
  let _ = fork (fun () ->
    watch timeout (fun () ->
      let v = await request in
      emit handled ();
      emit output (`Ok v))) in
  let _ = fork (fun () ->
    when_ timeout (fun () ->
      present_then_else handled
        (fun () -> ())
        (fun () -> emit output `Timeout))) in
  let v = await input in
  emit request v

let () = execute program
```

Step-by-step behavior:

(1) A timer process emits `timeout` after 3 instants.
(2) A forked branch waits on `request` but is wrapped in `watch timeout`, so it is terminated at the end of the instant in which `timeout` is emitted.
(3) If `request` arrives before timeout, the guarded branch emits `` `Ok v `` and marks `handled`. Otherwise, the timeout branch emits `` `Timeout ``.

Note: the `handled` guard prevents emitting `` `Timeout `` in the same instant as `` `Ok v ``, even under weak preemption.

In practice, this pattern makes timeouts compositional: the timeout branch is expressed separately from the request handler, while determinism is preserved through explicit guards and instant boundaries.

Running example used in the implementation section. The following small program appears again in the implementation section to illustrate queue movement, guard blocking, and weak preemption:

```
let g = new_signal () in
let s = new_signal () in
let guarded = (fun () -> when_ g (fun () -> emit s ())) in
let watched = (fun () -> watch s (fun () -> pause (); emit s
      ())) in
parallel [guarded; watched]
```

The key behaviors are: (1) the guarded task blocks until `g` is present; (2) the watched task can emit `s` in the same instant; (3) the `watch` prevents the watched task from continuing in the next instant.

Guard disjunctions and preemption. The following patterns illustrate that guard disjunctions are obtained by parallel guarded branches, and that nested preemptions act as a disjunction of stopping conditions.

```
(* Guard disjunction via parallel branches. *)
let guarded body =
  parallel [
    (fun () -> when_ g1 (fun () -> body ()));
    (fun () -> when_ g2 (fun () -> body ()))
```

```
  ]

(* Preemption disjunction: stop if s1 OR s2 is emitted. *)
let preempted body =
  watch s1 (fun () ->
    watch s2 (fun () ->
      body ()))
```

Derived control primitives. Tempo provides higher-level control structures built from the primitives. For instance, `present_then_else` branches on signal presence at the end of an instant. It provides a deterministic conditional construct.

```
(* Execute then_body if s becomes present this instant, else
      else_body. *)
present_then_else s
  (fun () -> then_body ())
  (fun () -> else_body ())
```

Another derived pattern is `every_do`, which executes a body whenever the signal becomes present, while avoiding immediate re-triggering within the same instant.

```
let rec every_do s body =
  let _ = await s in
  pause ();
  let completion = new_signal () in
  watch s (fun () ->
    body ();
    emit completion ());
  present_then_else completion
    (fun () -> pause (); every_do s body)
    (fun () -> every_do s body)
```

This pattern uses a `pause` to avoid strong re-triggering in the same instant, preserving weak restart semantics aligned with synchronous control. This corresponds to the restart discipline described in Section 4, where explicit pauses separate instants to keep reactions deterministic.

Use case: reactive I/O protocol. This example models a simple device handshake: a request from the environment must be acknowledged within a bounded number of instants. If the acknowledgement does not arrive in time, the protocol emits a timeout. This pattern is typical of sensor/actuator control loops where a response deadline is required. Here, the environment is the host input stream provided to `execute`. The device model is a simulated internal component used to illustrate the protocol; in a real system it would be replaced by actual I/O.

```
let rec timeout_after n timeout =
  if n = 0 then emit timeout ()
  else (pause (); timeout_after (n - 1) timeout)

let protocol request ack output =
  let timeout = new_signal () in
  let _ = fork (fun () -> timeout_after 2 timeout) in
  let handled = new_signal () in
  let _ = fork (fun () ->
    watch timeout (fun () ->
      let payload = await request in
      emit handled ();
      emit output (`Got payload))) in
  let _ = fork (fun () ->
    when_ timeout (fun () ->
      present_then_else handled
        (fun () -> ())
        (fun () -> emit output `Timeout))) in
  let _ = await ack in
  ()

let program input output =
  let ack = new_signal () in (* internal ack from device
       model *)
  let _ = fork (fun () -> protocol input ack output) in
  (* Environment-provided request; internal device emits ack
       . *)
  let _ = fork (fun () ->
    let _ = await input in
    pause ();
    emit ack ()) in
  ()

let () = execute program
```

Step-by-step behavior:

(1) Instant t: the protocol waits for `request` and arms a timer.
(2) Instant t+1: if `request` arrives, the simulated device emits `ack` in the next instant; the protocol can emit `'Got payload` unless the timeout fires first.
(3) Instant t+2: if no request was handled, the timeout branch emits `'Timeout`. The `watch` ensures the request handler is preempted once the timeout is present.

Instant	Observable events
t	Environment may emit `request`.
t+1	Internal: `ack` emitted by device model; protocol may emit `'Got payload`.
t+2	Internal: timeout may emit `'Timeout` if not handled.

Table 3: Timeline for the reactive I/O protocol.

Timeline (environment vs internal).

Extended scenario: periodic sampling with timeout. This example models a periodic sensor read where a sample must be acknowledged within a fixed number of instants; otherwise a timeout is raised.

```
let rec every_tick tick body =
  let _ = await tick in
  body ();
  pause ();
  every_tick tick body

let rec timeout_after n timeout =
  if n = 0 then emit timeout ()
```

```
  else (pause (); timeout_after (n - 1) timeout)

let program input output =
  let tick = new_signal () in
  let ack = new_signal () in
  let timeout = new_signal () in
  let _ = fork (fun () -> every_tick tick (fun () -> emit
       input ())) in
  let _ = fork (fun () -> timeout_after 2 timeout) in
  let _ = fork (fun () ->
    watch timeout (fun () ->
      let _ = await input in
      emit ack ())) in
  when_ timeout (fun () -> emit output `Timeout);
  let _ = await ack in
  emit output `Ok
```

This scenario highlights the interaction between periodic activation, weak preemption, and bounded response times.

7 ALGEBRAIC EFFECTS IN OCAML

This section provides background on effects and handlers; readers focused on Tempo can skip to the overview and return here for implementation details.

One-sentence intuition. An effect captures "what to do next" (the continuation), and the scheduler decides *when* to resume it, which aligns naturally with instant-based reactive execution. Algebraic effects provide a structured way to model effectful operations by separating the declaration of an effect from its interpretation. Instead of hardwiring side effects into the runtime or encoding them as monads, a program *performs* an effect at a call site, and a handler decides how to interpret it. The handler can resume the captured continuation immediately, defer it, or even resume it multiple times, which makes effects a natural fit for scheduling, suspension, and user-defined control operators.

In OCaml 5, effect handlers are first-class and integrate with the language runtime. The model follows the theory of algebraic effects and handlers [28], and it has been validated by practical systems such as Eff [5] and the Multicore OCaml project [16]. For our purposes, the important point is that the handler sees the point of suspension as a value—a *delimited continuation*—and can choose precisely when to resume it. Algebraic effects can therefore be viewed as a disciplined interface to delimited continuations, with the handler providing the delimiter [13, 18].

From the programmer's perspective, effects look like ordinary function calls with additional control semantics. For instance, an `Await` effect can be implemented so that it captures the continuation and stores it in a signal's waiter list, rather than resuming it immediately. A `Pause` effect can package the continuation into a task and enqueue it for the next instant. Similarly, `Emit` can be interpreted as a state update plus a resumption of waiting tasks. The key idea is that the operational behavior is defined in the handler, not at the call site.

This style is particularly well suited to synchronous reactive runtimes, where the semantics depend on *when* computations resume. By choosing whether a continuation is resumed in the current instant or deferred to the next one, the handler can implement the delayed-absence discipline and maintain determinism. In other words, algebraic effects provide the control lever that makes an instant-based scheduler possible at library level.

For Tempo, algebraic effects are therefore not an implementation detail but the core enabler: they allow the library to encode reactive constructs (signals, guards, preemption) directly as effectful operations. The scheduler lives in user space, with explicit queues and signal registries, and the effect handler defines the boundary between instantaneous reactions and deferred computation.

Deep vs shallow handlers. OCaml 5 distinguishes *deep* handlers, where the continuation is automatically re-wrapped by the same handler upon resumption, from *shallow* handlers, where the resuming code runs outside the handler unless it is re-installed explicitly. Tempo uses deep handlers (via `Effect.Deep`) to ensure that every resumed task continues under the same scheduler, which is essential for consistent instant semantics.

Cooperative threads via effects. It is well known that algebraic effects and delimited continuations can be used to implement cooperative threading. A `Yield` or `Pause` effect captures the current continuation and enqueues it, while the scheduler selects another task to run. This is the same pattern used in many effect-based thread libraries and serves as the foundation for Tempo's instant-based cooperative scheduler.

Intuition (small example). The following toy sketch shows how an effect is performed and how a handler decides when to resume the continuation.

```
(* Effect declaration *)
type _ Effect.t +=
  | Yield : unit Effect.t

let yield () = perform Yield

(* Handler decides when to resume *)
let run f =
  match f () with
  | () -> ()
  | effect Yield k ->
      (* Defer continuation; resume later *)
      Queue.add (fun () -> continue k ()) pending
```

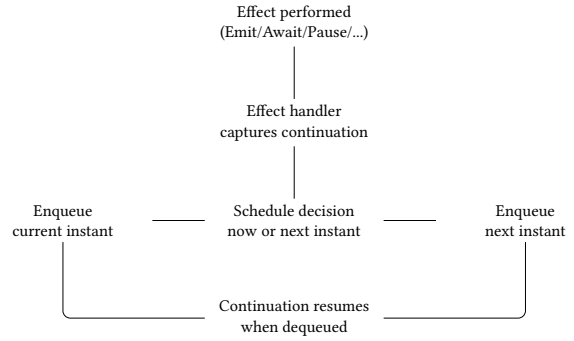

Figure 2: Effect handling and continuation scheduling.

In the next section, we show how Tempo uses these mechanisms to implement a synchronous runtime with explicit instants, signals, and control primitives.

8 IMPLEMENTATION WITH ALGEBRAIC EFFECTS (OCAML 5.3)

This section can be read independently; it connects the public API to the runtime mechanisms and effect handlers. Tempo implements its synchronous model as a handler-driven runtime. The design keeps the semantics explicit in library code while using OCaml 5.3 algebraic effects to capture and schedule continuations. This section focuses on how the effects-based implementation realizes the Tempo API.

Implementation map. For readers skimming the code, the implementation can be read in three passes: (1) the effect declarations in `lib/tempo_types.ml`; (2) the scheduler loop and task queues in `lib/tempo.ml`; (3) the handlers for each effect in the same file. The most important invariant to keep in mind is that all continuations are resumed by enqueueing tasks, never by direct calls, which preserves the instant discipline.

Scheduler state and task queues. The runtime maintains three task collections: a FIFO queue of tasks runnable in the current instant, a list of tasks scheduled for the next instant, and a list of tasks blocked on unsatisfied guards. Each task carries its guards and preemption tokens, and tasks are grouped into logical threads so that `join` can synchronize on thread completion. A registry of signals tracks presence, current value, and waiting continuations.

The scheduler state also stores debug counters (signal ids, task ids, step and instant counters) and a per-thread bookkeeping table. Each thread state keeps an active task count and a list of waiters to resume when the thread completes. This enables `join` to synchronize without global locks or OS threads.

The scheduler is single-threaded and deterministic: all decisions are made in one runtime loop. This makes it easier to reason about the ordering of effects within an instant, but it also means that the implementation must explicitly track every deferred continuation to avoid hidden control flow.

Tasks move between queues as follows: when a task performs an effect that defers it (e.g., `await` or `pause`), its continuation is wrapped as a new task and enqueued in `next_instant`. When a task is blocked by guards, it is placed in the `blocked` list and also registered in the missing signals' `guard_waiters`. At instant end, all blocked tasks are moved to `next_instant`, preserving the rule that absent guards only become observable in the next instant.

The scheduler also maintains a clear quiescence condition: an instant ends only when the current queue is empty. This ensures that all reactions to present signals have been taken before finalization, aligning with the delayed-absence semantics.

Task lifecycle and bookkeeping. Creating a task registers it under a thread id and increments the active count. When a task finishes, the active count is decremented; if it reaches zero, any waiters are resumed. Tasks carry a *queued* flag to avoid double enqueue, a *blocked* flag to control guard rescheduling, and a list of kill tokens that determine whether the task is still alive. This lifecycle is essential to the deterministic semantics because it ensures tasks are resumed only at well-defined instant boundaries.

Threads, `fork`*, and* `join`*.* A `fork` creates a new logical thread with its own task counter. The parent task continues immediately, while the child task is scheduled in the current instant. `join` captures the caller's continuation and registers it in the target thread's waiter list. When the thread's active counter reaches zero, all join waiters are resumed in the next instant. This design allows structured concurrency without OS threads, while still making completion observable at instant boundaries.

```
(* Pseudo-code for join handling *)
on Join thread_id:
  if thread completed then
    enqueue_next (continue k ())
  else
    add waiter (fun () -> enqueue_next (continue k ()))
```

Invariants. The scheduler maintains several invariants that are critical for correct behavior:

- **No direct resumption**: every continuation becomes a task and is enqueued through the scheduler queues.
- **No re-entrancy**: a task is never scheduled while it is already running (queued flag).
- **Guard discipline**: a blocked task appears in guard_waiters of every missing guard and nowhere else.
- **Kill discipline**: a task is scheduled only if all its kill tokens are alive.

Most of the implementation complexity exists to preserve these invariants in the presence of nested guards, nested preemption, and parallel composition.

Failure modes and debug hooks. Several runtime checks enforce the synchronous discipline:

- **Multiple emits on event signals**: a second emission in the same instant raises an exception because the event semantics require a single value per instant.
- **Present-without-value**: if a signal is marked present but has no stored value, the runtime fails fast to avoid undefined behavior.
- **Dead kill tokens**: tasks protected by aborted kill tokens are discarded silently; this is intentional and implements weak preemption.

Debug counters (task ids, signal ids, instant/step counters) and optional logs make it possible to correlate observable outputs with internal queue states. These hooks are crucial when diagnosing unexpected blocking or when confirming that a program respects the instant discipline.

Core runtime types. The implementation centers on a small set of runtime records. The following summary aligns with the concrete types in `lib/tempo_types.ml` and `lib/tempo.ml`.

Declared effects. The reactive operations are exposed as algebraic effects in `lib/tempo_types.ml`. The code below mirrors the actual declaration:

Type	Role and fields (informal)
task	t_id, thread, guards, kills, run, plus queued/blocked flags.
signal_core	s_id, present, value, awaiters, guard_waiters, and kind (event or aggregate with combine/initial).
thread_state	active, completed, waiters.
scheduler_state	current, next_instant, blocked, signals, thread_counter, threads, and debug counters.

Table 4: Key runtime structures used by the scheduler.

```
type _ Effect.t +=
  | New_signal : unit -> ('a, 'a, event) signal_core Effect.
      t
  | New_signal_agg :
      'agg * ('agg -> 'emit -> 'agg)
     -> ('emit, 'agg, aggregate) signal_core Effect.t
  | Emit : ('emit, 'agg, 'mode) signal_core * 'emit -> unit
      Effect.t
  | Await : ('emit, 'agg, 'mode) signal_core -> 'agg Effect.
      t
  | Await_immediate : ('a, 'a, event) signal_core -> 'a
      Effect.t
  | Pause : unit Effect.t
  | Fork : (unit -> unit) -> thread Effect.t
  | Join : thread -> unit Effect.t
  | With_guard :
      ('emit, 'agg, 'mode) signal_core * (unit -> unit) ->
          unit Effect.t
  | With_kill : kill * (unit -> unit) -> unit Effect.t
```

Each effect corresponds to a single scheduler action: `New_signal` allocates a fresh signal; `Emit` publishes a value and wakes waiters; `Await`/`Await_immediate` suspend and resume continuations; `Pause` shifts execution to the next instant; `Fork`/`Join` manage logical threads; and `With_guard`/`With_kill` implement guarding and weak preemption.

From effect to scheduler (worked example). Consider `Await` as implemented in `lib/tempo.ml`. When a task performs `Await s`, the handler captures the continuation and registers it in `s.awaiters`. If `s` is already present and is an event signal, the continuation is scheduled for the next instant; otherwise it is stored as a waiter and resumed when `s` becomes present. The critical behavior is that resumption is *always* scheduled through the scheduler queue, never executed directly in the current stack, which preserves the instant boundary discipline.

In the excerpt below, the helper `mk_task` wraps a continuation into a runnable task. `enqueue_next` schedules it for the next instant, and `continue k v` resumes the captured continuation with value `v`.

```
(* lib/tempo.ml: effect handling for Await *)
| effect (Await s), k ->
    let enqueue_resume v =
      let t' = mk_task st t.thread t.guards t.kills
        (fun () -> continue k v) in
      enqueue_next st t'
    in
    let register_waiter () =
      let resume v =
        let t' = mk_task st t.thread t.guards t.kills
          (fun () -> continue k v) in
        enqueue_next st t'
      in
      s.awaiters <- resume :: s.awaiters
    in
    if s.present then
      match s.kind with
      | Event_signal ->
          (match s.value with
           | Some v -> enqueue_resume v
           | None -> failwith "present but no value")
      | Aggregate_signal _ ->
          register_waiter ()
    else
      register_waiter ()
```

In terms of observable behavior, this means that an `await` never resumes in the current instant; the continuation is always deferred to the next instant, ensuring that presence and absence are observed only at instant boundaries. This pattern is representative of how effects encode the scheduler policy in user space.

Effect	Scheduler action and instant semantics
New_signal	Allocate signal state in the current instant.
Emit	Mark present and wake awaiters (possibly now).
Await	Register continuation; resume next instant.
Await_immediate	Resume now if present; otherwise register waiter.
Pause	Defer continuation to the next instant.
Fork	Spawn child task in current instant.
Join	Suspend until thread completes, then resume.
With_guard	Attach guard set; block if guards absent.
With_kill	Register cleanup; abort at instant end on kill.

Table 5: Effect-to-scheduler mapping.

Instant loop and signal finalization. Execution proceeds by instants. During an instant, the scheduler repeatedly dequeues a runnable task, checks whether its guards are satisfied, and either runs it or moves it to the blocked list. At the end of the instant, the runtime finalizes all signals: event and aggregate signals are reset to absent, guard waiters are cleared, and aggregate-signal awaiters are resumed with the accumulated value. Blocked tasks are moved to the next-instant list, and the next instant begins with the tasks that remain alive.

More precisely, each step records timing metrics, logs queue snapshots, and updates counters for reproducibility. When the runnable

queue becomes empty, the instant ends. All blocked tasks are re-queued for the next instant, which implements the delayed-absence discipline: a guard that was absent remains absent for the entire instant, and the task can only retry in the next one.

Re-queuing does not lose guard information: the blocked task still carries its guard list, so when it is dequeued in the next instant it is checked again. If some guards are still absent, it is re-registered in the corresponding `guard_waiters` lists. This is why clearing `guard_waiters` at instant end is safe: the lists are reconstructed as tasks are revisited.

Queue discipline and fairness. The current queue is FIFO, which yields a simple notion of fairness within an instant: tasks are resumed in the order they are enqueued. This ordering is an implementation choice rather than a semantic guarantee, and observable results should not depend on it. The ordering is nevertheless useful for debugging and reproducible traces, and it aligns with the cooperative scheduling model of FairThreads.

To avoid starvation across instants, any task that remains blocked is reconsidered in the next instant. This provides a coarse-grained fairness property: if a guard becomes present in some future instant, the corresponding task will be eligible to run at the start of that instant.

```
Instant loop pseudo-code
while current_queue not empty do
  t <- pop current_queue
  if not kills_alive t then discard t
  else if guard_ok t.guards then run t
  else
    block t on missing_guards
    register t in guard_waiters of each missing guard
end
finalize_signals ()
move blocked tasks to next_instant
swap current_queue and next_instant
```

This sketch omits bookkeeping details (thread counters, debug logs) but captures the control structure of the scheduler.

The ordering above is deliberate. Kill checks precede guard checks so that a preempted task is never allowed to run or block again. Guard checks precede execution to preserve the semantics that guards are blocking conditions, not filters applied after a step. If these checks were reversed, a task might run in an instant where its guard is absent or continue after being preempted, which would violate the intended weak-preemption semantics.

Execution trace (guard + watch). Consider the running example from Section 6. In instant t, one task is guarded by signal `g`, and one is wrapped in `watch s`. At the start of the instant, both tasks are in the current queue. The guarded task is blocked because `g` is absent and is registered in `g.guard_waiters`. The watched task runs; if it emits `s` during t, the guardian aborts the kill token and the watched task is prevented from continuing into t+1. At instant end, guard waiters are cleared, blocked tasks are moved to the next-instant list, and the system advances. This trace illustrates how guard blocking and weak preemption interact with the instant boundary discipline.

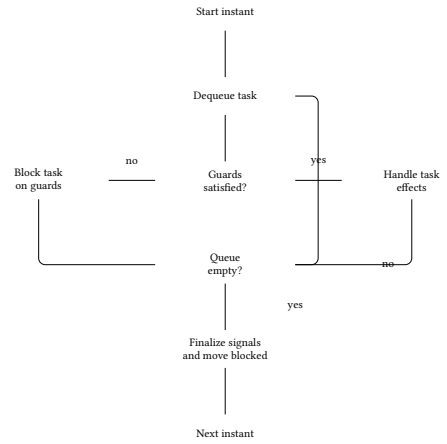

Figure 3: High-level flowchart of the instant-based scheduler.

```
let g = new_signal () in
let s = new_signal () in
let guarded = (fun () -> when_ g (fun () -> emit s ())) in
let watched = (fun () -> watch s (fun () -> pause (); emit s
      ())) in
parallel [guarded; watched]
```

The trace tables below show the corresponding queue, signal, and observable states across instants.

Instant	Queue/blocked/signals snapshot
t start	Current: `guarded`, `watched`; blocked: $\varnothing$; signals: $g = \bot$, $s = \bot$.
t mid	`guarded` blocked on g; `watched` runs and emits $s = ()$; kill token marked.
t end	Guard waiters cleared; blocked o next instant; s observed present at boundary, then reset; `watched` will not continue into t+1.

Table 6: Concrete execution trace for guard + watch (example above).

Instant	Observable signals and outputs
t	$s = ()$ becomes present due to `watched`; no output emitted.
t+1	`guarded` remains blocked if $g = \bot$; no output emitted.

Table 7: Two-instant observable trace for the example program.

Signal operations. Signal creation effects allocate runtime state and register signals in the global registry. `Emit` marks a signal present and stores its value. For event signals, multiple emissions in the same instant are rejected, while aggregate signals fold successive emissions into an accumulator. Awaiting a signal registers a continuation: `await` always resumes at the next instant, while

`await_immediate` can resume in the current instant when the event signal is already present. This distinction matches the delayed absence discipline while still allowing immediate reactions when safe. Figure 4 summarizes the signal state evolution within a single instant.

Event signals enforce a single-emission rule. The implementation checks `s.present` and raises an exception on a second emission in the same instant. This runtime check enforces the synchronous discipline in the absence of static typing rules.

Signal waiters are stored as callbacks (continuations wrapped as functions). For event signals, each emission registers resumption through the scheduler; `await` resumes in the next instant, while `await_immediate` may resume in the current instant if the signal is already present. For aggregate signals, emissions only update the accumulator; awaiters are resumed once at instant end with the aggregated value. This makes the point of observation explicit and keeps the aggregation order-insensitive.

Aggregate signals are delivered only once per instant. Emissions update the accumulator, but awaiters are resumed at instant end with the accumulated value. This preserves determinism because the order of emissions within the instant does not affect the observable schedule, only the aggregated result. If the combine function is not associative/commutative, the aggregate value may still depend on intra-instant evaluation order, which is intentionally left unspecified; this is why the API recommends pure, order-insensitive combines.

The finalization step can be summarized as follows:

```
for each signal s in registry do
  if s is aggregate then
    resume all awaiters with s.value
  clear s.awaiters and s.guard_waiters
  reset s.present and s.value
```

In the actual implementation, event-signal awaiters have already been resumed on emission, whereas aggregate awaiters are resumed in this finalization phase.

Guards and blocking. Guarded execution is implemented by attaching a list of guard signals to each task. If a guard is missing, the task is blocked and registered in the guard waiter lists of the missing signals. When a signal is emitted, its guard waiters are rechecked: tasks whose guards are now all present are moved back to the runnable queue, otherwise they remain blocked on the remaining missing guards. This implements the disjunctive behavior of nested guards while preserving determinism.

Guard checking is centralized: `guard_ok` verifies all guards are present, while `missing_guards` returns the absent subset. This keeps guard logic explicit and ensures the scheduler, rather than user code, controls when a blocked task can resume.

To avoid repeated scans, each signal maintains its own `guard_waiters` list. When a signal becomes present, only tasks that were blocked on that signal are reconsidered. Tasks may remain blocked if other guards are still absent, and they are re-registered on the remaining missing guards. This local indexing keeps guard rescheduling proportional to the number of blocked tasks that are directly affected by the emission.

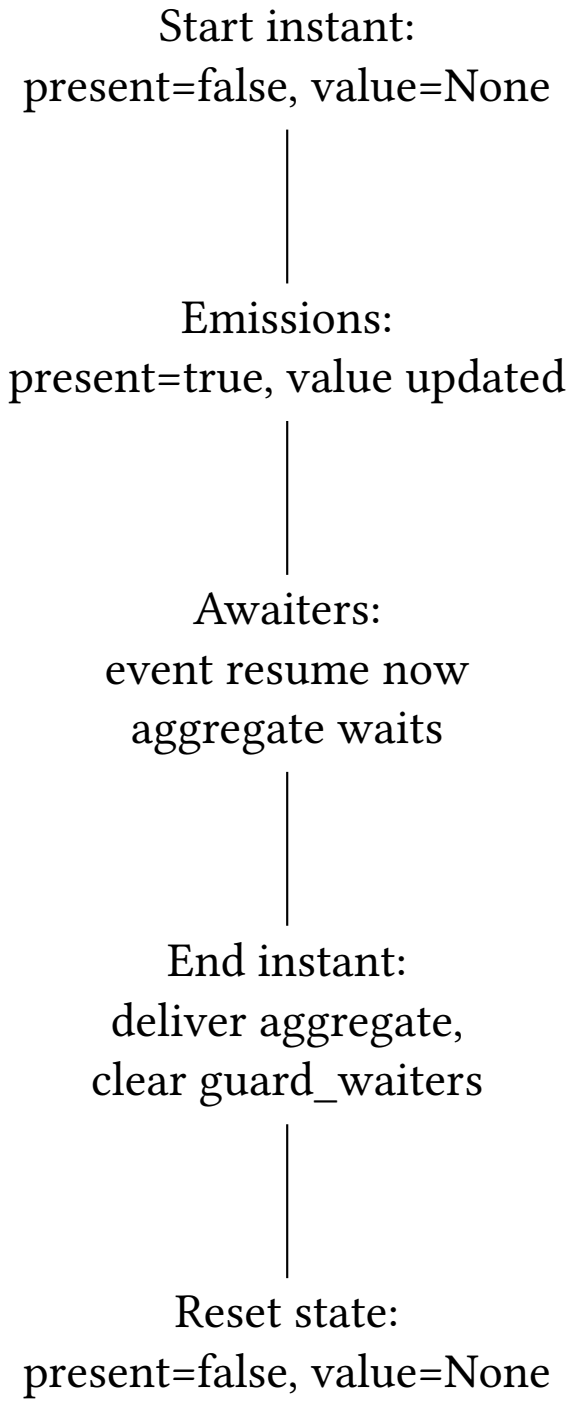

Figure 4: Signal state evolution within an instant.

Guard disjunction by nesting. Nested `when_` expressions behave as a conjunction at the level of guard satisfaction: a task blocked on multiple guards remains blocked until *all* guards are present in the same instant. If a program wants a logical disjunction (guard

g_1 *or* g_2), it must use parallel guarded branches, which creates distinct tasks each blocked on a single guard. This mechanism follows directly from the guard list representation and the centralized guard_ok predicate.

Effect `With_guard` *and* `when_`. The `when_` combinator is implemented using the `With_guard` effect. Performing `With_guard (s, body)` creates a new task whose guard list is extended with `s`. If `s` is absent, the task is blocked and registered in `s.guard_waiters`. Once `s` becomes present, the scheduler rechecks the guard list and enqueues the task in the current instant. This means that `when_` does not execute body until the guard is satisfied, and the scheduling decision remains centralized. In the excerpt below, `Any s` wraps a typed signal into an existential for the guard list, and `enqueue_now` schedules the guarded task in the current instant.

```
(* lib/tempo.ml: effect handling for With_guard *)
| effect (With_guard (s, body)), k ->
    let t' =
      mk_task st t.thread (Any s :: t.guards) t.kills
        (fun () ->
          body ();
          enqueue_now st (mk_task st t.thread t.guards t.
               kills
            (fun () -> continue k ())))
    in
    enqueue_now st t'
```

Guard resumption trace. Consider a task guarded by g_1 and g_2. At instant t, g_1 is present but g_2 is absent. The task is blocked and registered in g_2.*guard_waiters* only; it is not re-checked until g_2 becomes present. When g_2 is emitted in instant t+1, the scheduler revisits the task, finds all guards present, and enqueues it in the current queue. This preserves the intended "all guards in the same instant" semantics.

Effect `With_kill` *and* `watch`. Weak preemption relies on the `With_kill` effect. A kill token stores a cleanup continuation that is invoked when the token is aborted. The `watch` combinator creates a shared kill token for the protected body and a guardian task that waits on the watched signal. When the signal is emitted, the guardian aborts the shared token, which triggers the cleanup and prevents further resumption of the protected task.

Concretely, `watch` runs the body under `With_kill` and installs a cleanup continuation that schedules the parent continuation at the appropriate instant boundary. The scheduler checks `kills_alive` before enqueuing a task, so any task protected by an aborted kill token is discarded. This yields weak preemption: the body can finish the current instant, but it will not continue in subsequent instants once the watched signal is present. In the excerpt below, `continue_now` resumes the parent continuation in the current instant, while `continue_later` defers it to the next instant. The cleanup stored in `kk.cleanup` chooses the safe resumption point once the kill token is aborted.

```
(* lib/tempo.ml: effect handling for With_kill *)
| effect (With_kill (kk, body)), k ->
    let continue_now () =
      kk.cleanup <- None;
      enqueue_now st (mk_task st t.thread t.guards t.kills
        (fun () -> continue k ()))
    and continue_later () =
      kk.cleanup <- None;
      enqueue_next st (mk_task st t.thread t.guards t.kills
        (fun () -> continue k ()))
    in
    kk.cleanup <- Some continue_later;
    let runner () =
      body ();
      continue_now ()
    in
    let t' = mk_task st t.thread t.guards (kk :: t.kills)
         runner in
    enqueue_now st t'
```

Worked walkthrough: `watch`. Consider a body `B` under `watch s`. At instant t, the guardian awaits `s` while `B` runs. If `s` is emitted during t, the guardian aborts the shared kill token and installs cleanup to prevent `B` from continuing at t+1. If `s` is not emitted, `B` continues normally in the next instant. This aligns with weak preemption: the body can complete the current instant but is cut off before the next one.

Weak preemption with kill tokens. Weak preemption is modeled through kill tokens. A `watch` spawns a guardian that waits on the watched signal and aborts the protected computation at the end of the instant in which the signal is emitted. Internally, the runtime stores cleanup continuations for kill tokens so that aborting a token can schedule the appropriate continuation either immediately or in the next instant. This approach encodes preemption without violating the single-instant deterministic semantics.

Kill tokens also ensure that tasks are not resumed after preemption. The scheduler checks that all kill tokens are alive before enqueuing a task; if any token is dead, the task is discarded and the thread bookkeeping is updated. Because preemption is weak, application code sometimes adds an explicit guard (e.g., a `handled` signal) to prevent simultaneous emissions in the same instant; this pattern is illustrated in the usage examples.

Kill tokens and nested `watch`. Nested `watch` constructs install multiple kill tokens. The scheduler checks that *all* kill tokens are alive before running a task. This means any inner `watch` that aborts its token is sufficient to prevent the task from continuing, which realizes the intended disjunction of stopping conditions. Kill tokens do not cancel the current instant; they only prevent the continuation from being enqueued after the instant boundary, which preserves weak preemption.

Kill cleanup timing. The cleanup stored in a kill token is intentionally deferred with respect to observable behavior: if the watched signal is emitted in instant t, the protected computation can still finish its current instant but will not be resumed in t+1. Internally, the handler may schedule the cleanup either immediately or for the next instant, but the user-visible effect is always weak preemption at the instant boundary.

Host I/O hooks. The entry point `execute` creates dedicated input and output signals. Before each step, host input can emit on the input signal; after the step, the output signal is checked and delivered to the host. This keeps external I/O separate from synchronous reactions while preserving the instant structure.

Internally, `execute` installs a handler for all effects and runs a driver loop that alternates between (1) injecting host input for the instant, (2) running the scheduler until quiescence, and (3) collecting the output signal. This design keeps the host side free of continuation manipulation: the host only sees a stream of inputs and outputs at instant boundaries, which is consistent with the synchronous model.

```
loop:
  call input_hook to emit on input signal
  run scheduler until current queue is empty
  read output signal at instant end
```

Outputs are therefore observed only at instant boundaries, which matches the delayed-absence discipline and prevents mid-instant I/O from leaking implementation order.

For readability, helper functions used in the excerpts are summarized below (see Section 8).

Runtime helper glossary.

- `mk_task`: wrap a continuation into a runnable task record.
- `enqueue_now`: schedule a task in the current instant queue.
- `enqueue_next`: schedule a task for the next instant.
- `continue`: resume a captured continuation with a value.
- `Any`: existential wrapper to store signals in guard lists.
- `kills_alive`: check that all kill tokens of a task are alive.

8.1 Algebraic Effects in Detail

Tempo encodes the reactive primitives as algebraic effects and interprets them with a deep handler. Each effect represents a semantic action (e.g., emit, await, pause), and the handler decides whether to resume the continuation now or to re-schedule it for a later instant. This design makes the control flow explicit and keeps the scheduler in user space rather than in a custom runtime.

Effect capture preserves the current continuation, which is then wrapped into a task structure. For example, `await` captures the continuation, stores a resume function in the signal's waiter list, and ensures that the continuation is resumed at the beginning of the next instant. `await_immediate` selects between immediate resumption (if the signal is already present) and registration as a waiter (if it is not). Similarly, `pause` captures the continuation and places it in the next-instant list.

Signal creation effects allocate fresh signal records and return them to the continuation. `Emit` updates signal state and resumes any registered awaiters, potentially in the current instant. Guard effects update the task's guard set and may block the task if the guards are not satisfied. Preemption effects (`With_kill`) store cleanup continuations that are invoked when the kill token is aborted, ensuring that termination is coordinated with the instant boundary.

This effect-based organization provides a clear separation between the operational model (captured by the handler) and the user-facing API (exposed as effectful functions). It also enables experimentation with scheduling strategies without changing the OCaml runtime or the language semantics.

Handler pseudo-code

```
handle task =
  match task() with
  | effect (Emit (s, v)) k ->
      mark_present s v;
      resume_awaiters s;
      enqueue_now (continue k ())
  | effect (Await s) k ->
      register_waiter s (fun v -> enqueue_next (continue k v)
          )
  | effect (Await_immediate s) k ->
      if present s then enqueue_now (continue k (value s))
      else register_waiter s (fun v -> enqueue_now (continue
           k v))
  | effect Pause k ->
      enqueue_next (continue k ())
  | effect (With_guard (s, body)) k ->
      update_guards s;
      enqueue_now (fun () -> body (); continue k ())
  | effect (With_kill (kk, body)) k ->
      install_cleanup kk (fun () -> enqueue_next (continue k
           ()));
      enqueue_now (fun () -> body (); continue k ())
  | effect (Fork f) k ->
      enqueue_now f; enqueue_now (continue k ())
  | effect (Join t) k ->
      add_join_waiter t (fun () -> enqueue_now (continue k ()
          ))
```

8.2 Detailed Effect Traces

This subsection provides step-by-step traces for key primitives. The goal is to make explicit where continuations are stored and how they move across queues.

Trace: `await_immediate`. Suppose `s` is an event signal. If `s` is already present, the continuation is resumed in the current instant via `enqueue_now`; otherwise it is registered in `s.awaiters` for resumption at the next instant. The behavior is summarized below.

Case	Scheduler action
`s.present=true`	Enqueue continuation now; current instant resumes.
`s.present=false`	Store waiter; resume at next instant.

Table 8: Scheduler actions for `await_immediate`.

Trace: `pause`. `pause` captures the current continuation and enqueues it for the next instant. The task does not run again in the current instant, which is the primary mechanism for separating instants in user code.

Trace: `parallel`. `parallel` spawns multiple tasks in the current instant and waits for their completion. It relies on `fork` under the hood: each branch is a child task, and the parent waits until the

child thread's active count reaches zero. The effect-level view is therefore:

(1) Spawn a child task for each branch.
(2) Continue parent task to a `join`-like waiting point.
(3) Resume parent once all branches terminate.

Trace: `watch`. The following sketch shows the lifecycle of a watched task.

Step	Scheduler action
Start t	Spawn guardian awaiting s; run body task.
s emitted	Guardian aborts kill token; installs cleanup.
End t	Body may complete; continuation not enqueued for t+1.
t+1	Body is discarded if kill token is dead.

Table 9: Execution trace for `watch`.

Trace: `when_`. The guard handler creates a task with an extended guard list. If the guard is absent, the task is blocked; if the guard is present, the task is enqueued immediately. The observable result is that body only begins in an instant where the guard is present.

8.3 Code Map: from API to Runtime

The table below provides a roadmap from surface-level functions to the runtime mechanisms that implement them. This can be used as a guide when reading the source.

API function	Runtime mechanism
`new_signal`	`New_signal` effect, allocates signal_core.
`new_signal_agg`	`New_signal_agg` effect, sets combine/init.
`emit`	`Emit` effect, updates signal and wakes waiters.
`await`	`Await` effect, registers waiter for next instant.
`await_immediate`	`Await_immediate` effect, immediate or next.
`pause`	`Pause` effect, enqueue for next instant.
`when_`	`With_guard` effect, extend guard list.
`watch`	`With_kill` effect, install kill token/cleanup.
`parallel`	`fork`/`join`, thread counters.
`execute`	Scheduler loop + host I/O hooks.

Table 10: Mapping from API to runtime mechanisms.

9 PERFORMANCE

This section provides expected complexity and performance considerations in the absence of dedicated benchmarks.

Scheduler complexity. Let T be the number of runnable tasks in an instant, B the number of blocked tasks, and S the number of registered signals. The scheduler processes each runnable task at most once per step and checks guard satisfaction by scanning the task's guard list. Over an instant, the total cost is $O(T + G)$ where G is the sum of guard-list sizes across tasks, plus signal finalization at $O(S)$.

Signal operations. Emitting an event signal resumes all its awaiters immediately (in scheduler terms), so the cost is proportional to the number of waiters on that signal. Aggregate signals defer resumption to instant end, so emission cost is $O(1)$ plus the cost of the user-provided combine function. Finalization resumes all aggregate awaiters once per instant.

Queue sizes and memory. Memory usage grows with the number of tasks, signals, and registered waiters. In typical use, each `await` adds one continuation to a signal's waiter list, and each guard adds a task to a guard_waiter list. Worst-case memory consumption is therefore linear in the number of suspended tasks and guards.

Assumptions. These bounds assume: (1) guard lists are short; (2) signals are not created at unbounded rates per instant; (3) the combine function for aggregate signals is constant-time. Under these assumptions, the scheduler is expected to scale linearly with the number of active tasks and signals per instant.

Effect and allocation overhead. Each reactive primitive is implemented as an effect, which captures a continuation and allocates a task record. The dominant overheads are therefore continuation capture, task allocation, and queue operations. In practice, this means performance is sensitive to allocation rates and GC behavior, especially in programs that spawn many short-lived tasks per instant.

Preemption and guard costs. Weak preemption introduces a small constant overhead per protected task: a kill token, a cleanup closure, and a liveness check when the task is enqueued. Guard checks scale with the number of guard conditions attached to a task, which makes guard-heavy programs more sensitive to the size of guard lists than to the number of signals overall.

ReactiveML comparison plan. To compare Tempo against ReactiveML, we propose a small benchmark suite that captures the common synchronous patterns shared by both systems:

- **Ping-pong signals**: two processes alternately emit and await signals across instants (measures basic signaling and scheduling overhead).
- **Barrier synchronization**: N parallel tasks emit on a barrier signal and a coordinator waits for all emissions (measures aggregation and guard wakeups).
- **Timeout and preemption**: a request handler guarded by a timeout, implemented with weak preemption (measures watch/guard overhead).
- **Broadcast and fan-out**: one emitter wakes N awaiters in the same instant (measures awaiter resumption cost).
- **Guarded loops**: repeated guarded activation with pauses across many instants (measures long-running scheduling stability).

For each scenario, the metrics of interest are throughput (instants per second), latency per instant, and memory usage as a function of the number of tasks and signals. These benchmarks are intended as future work; the current draft does not include measured results.

Benchmark parameters.

Benchmark	Parameters and cost drivers
Ping-pong	I instants; cost dominated by signal wakeups.
Barrier	N tasks; cost dominated by guard wakeups and aggregation.
Timeout	I instants; cost dominated by watch/kill handling.
Broadcast	N awaiters; cost dominated by awaiter resumption.
Guarded loop	I instants; cost dominated by guard checks per instant.

Table 11: Benchmark parameters and expected cost drivers.

Reproducibility outline. Each scenario can be implemented as a standalone executable in `tests/ok` (Tempo) and a corresponding ReactiveML program, compiled with the ReactiveML toolchain. A minimal protocol should record: (1) number of instants executed; (2) wall-clock time; (3) task/signal counts. This makes the comparison transparent even before results are reported.

Concrete protocol. For each scenario, fix a parameter grid (e.g., number of tasks $N \in \{10, 10^2, 10^3\}$ and instants $I \in \{10^3, 10^4\}$), and run both Tempo and ReactiveML implementations on the same machine with CPU frequency scaling disabled. Measure wall-clock time for I instants, record peak resident memory, and log the number of signals and waiters. Report median over at least 5 runs.

Test coverage map. The list below shows which `tests/ok` programs exercise each public feature. This supports the claim that all features are covered by tests.

- **emit/await**: `emit_once_await_one.ml` `emit_once_await_many.ml`.
- **await_immediate**: `emit_once_await_immediate.ml`, `when_await_imm_same_instant.ml`, `when_await_imm_next_instant.ml`.
- **pause**: `pause.ml`.
- **parallel**: `parallel.ml`, `when_parallel.ml`.
- **guards/when_**: `when_next_instant.ml`, `when_same_instant.ml`, `when_nested.ml`, `when_complete.ml`.
- **watch/weak preemption**: `watch_complete.ml`, `watch_nested.ml`, `watch_killed.ml`, `watch_when_complete.ml`, `watch_when_killed.ml`.
- **kill**: `with_kill_abort.ml`, `with_kill_inherit.ml`.
- **aggregate signals**: `emit_agg_collect.ml` `emit_agg_sum.ml`.
- **present_then_else**: `present_then_else.ml`.

Testing strategy (summary). Tests are organized by primitive (signals, guards, preemption) and by composition patterns (parallel, nested guards, immediate awaits). Each test has an expected output, enabling regression checks on observable behavior at instant boundaries.

10 LIMITATIONS AND TRADE-OFFS

Tempo is designed as a lightweight library-level runtime. This choice comes with several limitations that define the current scope:

- **No static guarantees.** Determinism and correct use of primitives (e.g., single emission on event signals) are enforced at runtime rather than by a type system or compiler checks.
- **Runtime errors for misuse.** Multiple emissions on an event signal raise an exception; misuse must be caught by tests or careful discipline.
- **Effects require OCaml 5.3.** The approach relies on algebraic effects and deep handlers, which are not available in earlier versions.
- **Library-level scheduling only.** Tempo does not integrate with a language-level optimizer, so certain static analyses or compile-time optimizations (as in ReactiveML) are unavailable.
- **Low-level API risks.** The `Tempo.Low_level` module exposes primitives that can break high-level invariants if used incorrectly.
- **Determinism vs evaluation order.** Reference-based communication can make behavior depend on evaluation order within an instant, which is not fixed by the semantics; mitigating this requires static analyses (see Section 12).
- **Purity assumptions.** Aggregate signals assume order-insensitive combine functions; if combine is not associative/commutative, aggregate results may depend on intra-instant scheduling.
- **Logical parallelism only.** Concurrency is logical and single-core; enabling physical parallelism would require additional determinism controls (see Section 12).
- **Non-guaranteed behaviors.** Programs relying on shared mutable state across parallel tasks can observe different results even with identical inputs, because the intra-instant order is unspecified.

11 RELATED WORK

Tempo is rooted in the reactive control lineage: Esterel, FairThreads, and ReactiveML. It inherits Boussinot's delayed-absence discipline [9, 10, 23]. Relative to Esterel, Tempo does not impose a static compilation discipline; instead it targets dynamic behavior and library-level composition. Relative to FairThreads, Tempo provides a typed OCaml API with higher-order combinators and valued signals.

Other synchronous languages in the same family include Lustre and Signal, which take dataflow-oriented approaches to synchronous programming, and the broader survey literature that situates these languages in a shared semantic landscape [6, 19, 22]. Tempo aligns with this lineage but focuses on control primitives realized at library level rather than a dedicated compiler.

Industrial synchronous formalisms have also shaped this landscape. Statecharts provide a widely adopted visual model for reactive control [20], and SCADE offers a synchronous design and validation environment for safety-critical embedded control software [8]. These systems emphasize predictable control behavior and tooling, whereas Tempo targets a lightweight, library-level embedding.

Closely related academic languages include Lucid Synchrone and Quartz, which extend synchronous ideas with functional or model-based design perspectives [11, 29]. These reinforce the continuity between Tempo and earlier synchronous language research.

ReactiveML is the closest antecedent: it integrates reactive constructs into a language extension with dedicated syntax and compilation support. Tempo, by contrast, is a pure library that relies on OCaml 5.3 algebraic effects to implement the same control primitives without modifying the compiler or the runtime. This makes Tempo easier to integrate into existing OCaml codebases and enables experimentation with scheduling policies in user space, at the cost of foregoing some static analyses and language-level guarantees provided by ReactiveML.

In contrast with FRP [17], Tempo emphasizes control and instant-based determinism rather than continuous dataflow. The library focuses on explicit signals, guards, and preemption, aligning with synchronous reactive semantics instead of stream processing abstractions.

Stream-processing languages such as Lucid and StreamIt [1, 30] emphasize dataflow pipelines rather than control-centric instants. They provide a useful contrast to Tempo: the dataflow style is oriented toward throughput and steady-state streaming, while Tempo targets explicit control reactions at discrete instants.

Recent FRP systems have introduced stronger operational guarantees. The modal FRP line of work uses modal types to enforce causality and prevent space leaks, including Simply RaTT and its extensions [2–4]. These systems target dataflow FRP rather than control-oriented primitives, but they offer insights into static enforcement mechanisms that could complement Tempo's current runtime checks. Recent applications of modal FRP include GUI programming with modal types, which demonstrates the practicality of these systems in reactive user interfaces [15].

Industrial reactive-streams frameworks (e.g., RxJava, Akka Streams) emphasize asynchronous stream processing and backpressure rather than synchronous instants. While their semantics differ from Tempo's model, they represent a separate lineage of reactive programming practice and are surveyed in the reactive-streams literature [12, 14].

Tempo also builds on the recent availability of algebraic effects and handlers in OCaml [16]. At a semantic level, effects provide a structured interface to delimited continuations [13, 18, 28], which has long been recognized as a natural foundation for user-level schedulers and cooperative threads. Systems such as Eff demonstrated the practicality of effect handlers as a runtime control mechanism [5], and Tempo leverages the same mechanism for synchronous control, rather than for general-purpose effects.

Compared with effect-based concurrency libraries, Tempo's distinctive contribution is its *instant* discipline: continuations are not resumed arbitrarily, but at well-defined boundaries that expose synchronous semantics. This positions Tempo between language-level synchronous systems (Esterel, ReactiveML) and general-purpose effect-based schedulers, offering a library path to control-oriented reactive programming.

12 CONCLUSION

This draft presented Tempo as a lightweight synchronous runtime and explained how its control constructs are implemented with OCaml 5.3 algebraic effects. The report positioned Tempo within the reactive control lineage, clarified the core constructs (logical instants, signals, guards, weak preemption), and detailed the scheduler and effect-handler design. The current level of confidence in correctness relies on an extensive test suite that exercises the expected instant-based semantics across all public features (signals, aggregate signals, guards, preemption, await_immediate, parallel composition, and derived combinators). See Section 9 for a feature-to-test mapping.

The determinism guarantees discussed in this report assume standard purity discipline: programs should avoid shared mutable state whose outcomes depend on intra-instant scheduling, and aggregate combine functions should be order-insensitive. These assumptions match the synchronous control model and clarify the scope of the current guarantees.

Future work. We plan to formalize correctness in two steps. First, we will define a high-level synchronous semantics and prove a refinement relation with an abstract scheduler model (e.g., along the lines of formal treatments of synchronous languages and their operational models [7, 9, 24]). Second, we will connect this abstract scheduler to the concrete effect-handler implementation, drawing on formal work on algebraic effects and handlers [5, 28]. This two-layer approach is intended to bridge the gap between the user-facing semantics and the low-level execution mechanism.

Two additional concerns are central. First, the use of references for communication can break semantic determinism: even if the implementation fixes the evaluation order within an instant, that order is not specified by the semantics and can change across versions. Addressing this requires static analyses that detect or rule out order-sensitive behaviors. Second, Tempo's parallelism is currently logical rather than physical. Supporting physical parallelism risks reintroducing nondeterminism unless the scheduler enforces a deterministic memory discipline. We plan to investigate recent work by Pouzet and Mendler on PSM for guidance on deterministic shared-memory parallelism [26] (currently a preprint). Finally, adding asynchronous computations for selected programming idioms is an open direction, provided their interaction with instant-based determinism remains controlled.

Additional avenues include: (1) formalizing the host I/O contract with a precise notion of input injection and output observation per instant, aligned with synchronous language interfaces [9, 19]; (2) designing static analyses or effect disciplines to detect order-sensitive use of shared mutable state, in the spirit of clock/causality analyses [9, 19]; (3) exploring partial compilation or macro expansion to reduce effect overhead in hot paths, following compilation strategies in synchronous languages [9]; (4) adding instant-level profiling support (queue sizes, wakeups, guard checks) to aid performance diagnosis, building on tooling experience from reactive implementations [25]; and (5) clarifying aggregation semantics for non-commutative combines, either by fixing a reduction order or by restricting such combines to controlled modes, drawing on deterministic stream models [21].

As a simple illustration, two parallel tasks that both mutate a shared reference and then emit a signal can produce different outcomes depending on their evaluation order within the instant. Since that order is not part of the semantic contract, relying on it

compromises determinism and motivates static analyses that forbid or control such patterns.

```
let r = ref 0 in
parallel [
  (fun () -> r := 1; emit s !r);
  (fun () -> r := 2; emit s !r)
]
```

Depending on the evaluation order in the instant, the emitted value can be 1 or 2, despite identical source code.

REFERENCES

[1] E. A. Ashcroft and W. W. Wadge. 1977. Lucid, a Nonprocedural Language with Iteration. *Commun. ACM* 20, 7 (1977), 519–526. https://doi.org/10.1145/359636.359715

[2] Patrick Bahr. 2022. Modal FRP for All: Functional Reactive Programming without Space Leaks in Haskell. *Journal of Functional Programming* 32 (2022). https://doi.org/10.1017/S0956796822000132

[3] Patrick Bahr, Christian Uldal Graulund, and Rasmus Ejlers Møgelberg. 2019. Simply RaTT: A Fitch-Style Modal Calculus for Reactive Programming without Space Leaks. *Proceedings of the ACM on Programming Languages* 3, ICFP (2019), 1–27. https://doi.org/10.1145/3341713

[4] Patrick Bahr and Rasmus Ejlers Møgelberg. 2023. Asynchronous Modal FRP. *Proceedings of the ACM on Programming Languages* 7, ICFP (2023), 476–510. https://doi.org/10.1145/3607847

[5] Andrej Bauer and Matija Pretnar. 2015. Programming with Algebraic Effects and Handlers. *Journal of Logical and Algebraic Methods in Programming* 84, 1 (2015), 108–123. https://doi.org/10.1016/j.jlamp.2014.02.001

[6] Albert Benveniste, Paul Caspi, Stephen A. Edwards, Nicolas Halbwachs, Paul Le Guernic, and Roberto de Simone. 2003. The Synchronous Languages 12 Years Later. *Proc. IEEE* 91, 1 (2003), 64–83. https://doi.org/10.1109/JPROC.2002.805826

[7] Gérard Berry. 1985. The ESTEREL Synchronous Programming Language and Its Mathematical Semantics. In *Lecture Notes in Computer Science*. https://doi.org/10.1007/3-540-15670-4_19 Add exact volume/pages if needed.

[8] Gérard Berry. 2007. *SCADE: Synchronous Design and Validation of Embedded Control Software*. 19–33. https://doi.org/10.1007/978-1-4020-6254-4_2

[9] Gérard Berry and Georges Gonthier. 1992. The Esterel Synchronous Programming Language: Design, Semantics, Implementation. *Science of Computer Programming* 19, 2 (1992), 87–152. https://doi.org/10.1016/0167-6423(92)90005-V

[10] Frédéric Boussinot. 2005. FairThreads: Mixing Cooperative and Preemptive Threads in C. *Concurrency and Computation: Practice and Experience* 18, 5 (2005), 445–469. https://doi.org/10.1002/cpe.919

[11] Paul Caspi, Grégoire Hamon, and Marc Pouzet. 2008. Synchronous Functional Programming with Lucid Synchrone. , 207–247 pages. https://doi.org/10.1002/9780470611012.ch7

[12] Fu Cheng. 2017. *Reactive Streams*. 87–97. https://doi.org/10.1007/978-1-4842-3330-6_7

[13] Olivier Danvy and Andrzej Filinski. 1990. Abstracting Control. In *Proceedings of the 1990 ACM Conference on LISP and Functional Programming (LFP)*. 151–160. https://doi.org/10.1145/91556.91622

[14] Adam L. Davis. 2019. *Reactive Streams in Java: Concurrency with RxJava, Reactor, and Akka Streams*. https://doi.org/10.1007/978-1-4842-4176-9

[15] Jean-Claude Disch, Asger Heegaard, and Patrick Bahr. 2025. Functional Reactive GUI Programming with Modal Types. In *Trends in Functional Programming*. 93–114. https://doi.org/10.1007/978-3-031-99751-8_5

[16] Stephen Dolan, Leo White, Anil Madhavapeddy, et al. 2021. Retrofitting Effect Handlers onto OCaml. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*. 206–221. https://doi.org/10.1145/3453483.3454039

[17] Conal Elliott and Paul Hudak. 1997. Functional Reactive Animation. In *Proceedings of the International Conference on Functional Programming (ICFP)*. 263–273. https://doi.org/10.1145/258949.258973

[18] Andrzej Filinski. 1994. Representing Monads. In *Proceedings of the 21st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 446–457. https://doi.org/10.1145/174675.178047

[19] Nicolas Halbwachs, Paul Caspi, Pascal Raymond, and Daniel Pilaud. 1991. The Synchronous Data Flow Programming Language LUSTRE. *Proc. IEEE* 79, 9 (1991), 1305–1320. https://doi.org/10.1109/5.97300

[20] David Harel. 1987. Statecharts: A Visual Formalism for Complex Systems. *Science of Computer Programming* 8, 3 (1987), 231–274. https://doi.org/10.1016/0167-6423(87)90035-9

[21] Gilles Kahn. 1974. The Semantics of a Simple Language for Parallel Programming. In *Proceedings of IFIP Congress*. 471–475.

[22] Paul Le Guernic, Albert Benveniste, Paul Bournai, and Thierry Gautier. 1987. Synchronous Data Flow Programming with the Language SIGNAL. *IFAC Proceedings Volumes* 20, 2 (1987), 359–364. https://doi.org/10.1016/S1474-6670(17)55987-X

[23] Louis Mandel and Marc Pouzet. 2005. ReactiveML: A Reactive Extension to ML. In *Proceedings of the 7th ACM SIGPLAN International Conference on Principles and Practice of Declarative Programming (PPDP)*. 82–93. https://doi.org/10.1145/1069774.1069782

[24] Louis Mandel and Marc Pouzet. 2008. ReactiveML, un langage fonctionnel pour la programmation réactive. *Techniques et sciences informatiques* 27, 9-10 (2008), 1097–1128. https://doi.org/10.3166/tsi.27.1097-1128

[25] Louis Mandel, Marc Pouzet, and Luc Maranget. 2015. ReactiveML, Ten Years Later. In *Proceedings of the 17th ACM SIGPLAN International Symposium on Principles and Practice of Declarative Programming (PPDP)*. https://doi.org/10.1145/2790449.2790509 Add exact pages if needed.

[26] Michael Mendler and Marc Pouzet. 2025. PSM: Policy Synchronised Deterministic Memory. *CoRR* abs/2506.15424 (2025). https://doi.org/10.48550/ARXIV.2506.15424

[27] Ivan Perez, Manuel Bärenz, and Henrik Nilsson. 2016. Functional Reactive Programming, Refactored. In *Proceedings of the 9th International Symposium on Haskell*. 33–44. https://doi.org/10.1145/2976002.2976010

[28] Gordon Plotkin and Matija Pretnar. 2009. Handlers of Algebraic Effects. In *European Symposium on Programming (ESOP) (Lecture Notes in Computer Science, Vol. 5502)*. 80–94. https://doi.org/10.1007/978-3-642-00590-9_7

[29] Klaus Schneider and Jens Brandt. 2017. *Quartz: A Synchronous Language for Model-Based Design of Reactive Embedded Systems*. 29–58. https://doi.org/10.1007/978-94-017-7267-9_3

[30] William Thies, Michal Karczmarek, and Saman Amarasinghe. 2002. *StreamIt: A Language for Streaming Applications*. 179–196. https://doi.org/10.1007/3-540-45937-5_14